

LAPORAN PRAKTIKUM

STRATEGI ALGORITMA

PRAKTIKUM 4: ALGORITMA DIVIDE & CONQUER

I. DEFINISI

1.1 Algoritma Divide and Conquer

Definisi: Divide and Conquer adalah paradigma perancangan algoritma yang memecah masalah berukuran n menjadi beberapa upa-masalah yang lebih kecil, menyelesaikan setiap upa-masalah secara rekursif, lalu menggabungkan solusinya menjadi solusi masalah semula.

Tiga Tahap:

1. **Divide:** membagi masalah menjadi beberapa upa-masalah dengan ukuran lebih kecil dan karakteristik yang sama.
2. **Conquer:** menyelesaikan setiap upa-masalah secara rekursif hingga mencapai basis (ukuran cukup kecil untuk diselesaikan langsung).
3. **Combine:** menggabungkan solusi upa-masalah menjadi solusi masalah semula.

Fungsi: Digunakan untuk menyelesaikan persoalan yang dapat dibagi secara alami menjadi sub-masalah serupa, seperti pengurutan, pencarian minimum-maksimum, perkalian matriks, dan fast Fourier transform.

Kelebihan: Kompleksitas lebih efisien dibanding pendekatan iteratif biasa (contoh: Merge Sort $O(n \log n)$ vs $O(n^2)$), paralel secara alami karena upa-masalah independen, dan solusinya elegan secara rekursif.

Kekurangan: Overhead rekursi (stack calls) untuk masalah berukuran kecil, penggunaan memori tambahan pada tahap combine (contoh: Merge Sort membutuhkan $O(n)$ memori tambahan).

1.2 MinMaks Divide and Conquer

Mencari nilai minimum dan maksimum sekaligus pada tabel berisi n elemen dengan membagi tabel menjadi dua bagian di titik tengah, mencari MinMaks setiap bagian secara rekursif, lalu membandingkan hasil keduanya. Kompleksitas: $T(n) = O(n)$.

1.3 Merge Sort

Algoritma pengurutan berbasis D&C dengan tiga langkah: (a) Divide: bagi tabel menjadi dua bagian berukuran $n/2$, (b) Conquer: urutkan tiap bagian secara rekursif, (c) Merge: gabungkan dua bagian terurut menjadi satu tabel terurut. Kompleksitas: $T(n) = O(n \log n)$ di semua kasus. Bersifat stabil.

1.4 Quick Sort dan Partisi

Algoritma pengurutan D&C dengan pendekatan hard split/easy join. Tabel dipartisi menggunakan elemen pivot sehingga elemen di kiri pivot lebih kecil dan di kanan lebih besar, kemudian setiap bagian diurutkan secara rekursif tanpa tahap combine eksplisit. Kompleksitas: average $O(n \log n)$, worst case $O(n^2)$ apabila pivot selalu ekstrem. Tidak stabil.

II. PRE TEST

Pertanyaan 1: Buatlah fungsi QuickSort!

```
def quick_sort(arr, i, j):
    # basis: array 1 elemen atau kosong
    if i >= j:
        return

    # partisi dan dapatkan indeks pembagi
    k = partisi(arr, i, j)

    # rekursif urutkan bagian kiri dan kanan
    quick_sort(arr, i, k)
    quick_sort(arr, k + 1, j)

if __name__ == "__main__":
    data = [4, 12, 23, 9, 21, 1, 35, 2, 24]
    print("Data awal:", data)

    # MinMaks D&C
    mn, mx = minmaks(data, 0, len(data) - 1)
    print(f"\nMinMaks D&C -> Min: {mn}, Maks: {mx}")

    # Merge Sort
    arr_ms = data.copy()
    merge_sort(arr_ms, 0, len(arr_ms) - 1)
    print(f"Merge Sort -> {arr_ms}")

    # Quick Sort
    arr_qs = data.copy()
```

```
quick_sort(arr_qs, 0, len(arr_qs) - 1)
print(f"Quick Sort -> {arr_qs}")
```

Pertanyaan 2: Buatlah fungsi Partition!

```
def partisi(arr, i, j):
    # pilih elemen tengah sebagai pivot
    pivot = arr[(i + j) // 2]
    p, q = i, j

    while True:
        # scan dari kiri sampai menemukan elemen >= pivot
        while arr[p] < pivot:
            p += 1
        # scan dari kanan sampai menemukan elemen <= pivot
        while arr[q] > pivot:
            q -= 1

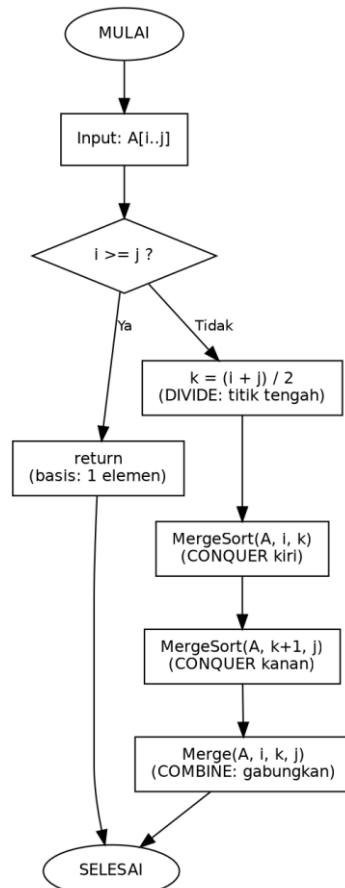
        # jika belum bertemu, tukar dan geser indeks
        if p < q:
            arr[p], arr[q] = arr[q], arr[p]
            p += 1
            q -= 1
        else:
            break

    return q
```

III. ANALISIS PRAKTIKUM

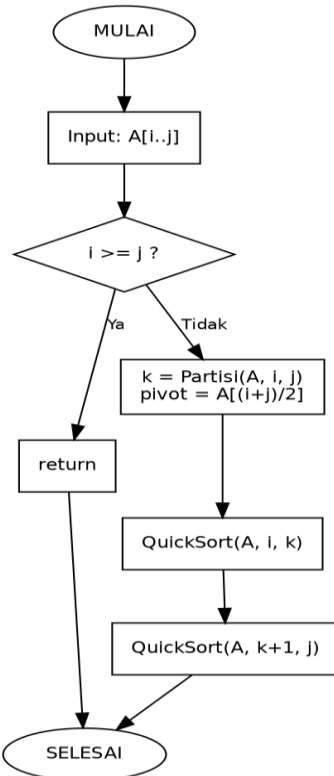
3.1 Langkah Praktikum: Program Sorting

Flowchart Merge Sort



Gambar 3.1 Flowchart Algoritma Merge Sort

Flowchart Quick Sort



Gambar 3.2 Flowchart Algoritma Quick Sort

Source Code Python: MinMaks D&C

```
def minmaks(arr, i, j):
    # basis: 1 elemen
    if i == j:
        return arr[i], arr[i]

    # basis: 2 elemen, bandingkan langsung
    if i == j - 1:
        if arr[i] < arr[j]:
            return arr[i], arr[j]
        else:
            return arr[j], arr[i]

    # rekursif: bagi tabel di titik tengah
    k = (i + j) // 2
    min1, maks1 = minmaks(arr, i, k)
    min2, maks2 = minmaks(arr, k + 1, j)

    # combine: ambil min dan maks dari kedua bagian
    return min(min1, min2), max(maks1, maks2)
```

Source Code Python: Merge Sort

```
def merge(arr, kiri, tengah, kanan):
    # salin dua bagian ke array sementara
    kidal1 = arr[kiri:tengah + 1]
    kidal2 = arr[tengah + 1:kanan + 1]

    i = j = 0
    k = kiri

    # gabungkan dua bagian secara terurut
    while i < len(kidal1) and j < len(kidal2):
        if kidal1[i] <= kidal2[j]:
            arr[k] = kidal1[i]
            i += 1
        else:
            arr[k] = kidal2[j]
            j += 1
        k += 1

    # salin sisa elemen kiri jika ada
    while i < len(kidal1):
        arr[k] = kidal1[i]
        i += 1
        k += 1

    # salin sisa elemen kanan jika ada
    while j < len(kidal2):
        arr[k] = kidal2[j]
        j += 1
        k += 1

def merge_sort(arr, i, j):
    # basis: array 1 elemen sudah terurut
    if i >= j:
        return

    # divide: tentukan titik tengah
    k = (i + j) // 2

    # conquer: urutkan bagian kiri dan kanan secara rekursif
    merge_sort(arr, i, k)
    merge_sort(arr, k + 1, j)

    # combine: gabungkan hasil pengurutan
    merge(arr, i, k, j)
```

```

def merge_sort(arr, i, j):
    # basis: array 1 elemen sudah terurut
    if i >= j:
        return

    # divide: tentukan titik tengah
    k = (i + j) // 2

    # conquer: urutkan bagian kiri dan kanan secara rekursif
    merge_sort(arr, i, k)
    merge_sort(arr, k + 1, j)

    # combine: gabungkan hasil pengurutan
    merge(arr, i, k, j)

```

Source Code Python: Quick Sort

```

def partisi(arr, i, j):
    # pilih elemen tengah sebagai pivot
    pivot = arr[(i + j) // 2]
    p, q = i, j

    while True:
        # scan dari kiri sampai menemukan elemen >= pivot
        while arr[p] < pivot:
            p += 1
        # scan dari kanan sampai menemukan elemen <= pivot
        while arr[q] > pivot:
            q -= 1

        # jika belum bertemu, tukar dan geser indeks
        if p < q:
            arr[p], arr[q] = arr[q], arr[p]
            p += 1
            q -= 1
        else:
            break

    return q

def quick_sort(arr, i, j):
    # basis: array 1 elemen atau kosong
    if i >= j:
        return

    # partisi dan dapatkan indeks pembagi
    k = partisi(arr, i, j)

    # rekursif urutkan bagian kiri dan kanan
    quick_sort(arr, i, k)
    quick_sort(arr, k + 1, j)

if __name__ == "__main__":
    data = [4, 12, 23, 9, 21, 1, 35, 2, 24]
    print("Data awal:", data)

    # MinMaks D&C
    mn, mx = minmaks(data, 0, len(data) - 1)
    print(f"\nMinMaks D&C -> Min: {mn}, Maks: {mx}")

    # Merge Sort
    arr_ms = data.copy()
    merge_sort(arr_ms, 0, len(arr_ms) - 1)
    print(f"Merge Sort -> {arr_ms}")

    # Quick Sort
    arr_qs = data.copy()
    quick_sort(arr_qs, 0, len(arr_qs) - 1)
    print(f"Quick Sort -> {arr_qs}")

```

Analisis Output

Program dijalankan dengan data awal [4, 12, 23, 9, 21, 1, 35, 2, 24]:

Algoritma	Output	Keterangan
MinMaks D&C	Min: 1, Maks: 35	Ditemukan dengan $T(n) = O(n)$ perbandingan
Merge Sort	[1, 2, 4, 9, 12, 21, 23, 24, 35]	Terurut stabil, $O(n \log n)$
Quick Sort	[1, 2, 4, 9, 12, 21, 23, 24, 35]	Terurut, rata-rata $O(n \log n)$

MinMaks D&C: Tabel dibagi dua pada $k=4$. Bagian kiri [4,12,23,9,21] menghasilkan $\text{min}=4$, $\text{maks}=23$. Bagian kanan [1,35,2,24] menghasilkan $\text{min}=1$, $\text{maks}=35$. Setelah combine: $\text{min}=1$, $\text{maks}=35$.

Merge Sort: Pembagian berlangsung rekursif hingga setiap bagian berukuran 1 elemen, lalu digabung secara terurut. Karena bersifat stabil, urutan relatif elemen bernilai sama terjaga.

Quick Sort: Pivot dipilih dari elemen tengah setiap iterasi. Proses partisi memindahkan elemen lebih kecil ke kiri dan lebih besar ke kanan pivot, kemudian kedua bagian diurutkan rekursif.

3.2 Post Test: Merge Sort Standalone

Source Code Python

```
def merge(arr, kiri, tengah, kanan):
    # salin dua bagian ke array sementara
    bagian_kiri = arr[kiri:tengah + 1]
    bagian_kanan = arr[tengah + 1:kanan + 1]

    i = j = 0
    k = kiri

    # gabungkan dua bagian ke array utama secara terurut
    while i < len(bagian_kiri) and j < len(bagian_kanan):
        if bagian_kiri[i] <= bagian_kanan[j]:
            arr[k] = bagian_kiri[i]
            i += 1
        else:
            arr[k] = bagian_kanan[j]
            j += 1
        k += 1

    # salin sisa bagian kiri jika masih ada
    while i < len(bagian_kiri):
        arr[k] = bagian_kiri[i]
        i += 1
        k += 1

    # salin sisa bagian kanan jika masih ada
    while j < len(bagian_kanan):
        arr[k] = bagian_kanan[j]
        j += 1
        k += 1
```

```

def merge_sort(arr, i, j, depth=0):
    indent = "  " * depth

    # basis: array dengan 1 elemen sudah terurut
    if i >= j:
        return

    # divide: hitung titik tengah pembagi
    k = (i + j) // 2
    print(f"{indent}Divide  [{i}..{j}] -> kiri [{i}..{k}], kanan [{k+1}..{j}]]")

    # conquer: urutkan bagian kiri secara rekursif
    merge_sort(arr, i, k, depth + 1)

    # conquer: urutkan bagian kanan secara rekursif
    merge_sort(arr, k + 1, j, depth + 1)

    # combine: gabungkan hasil pengurutan kedua bagian
    merge(arr, i, k, j)
    print(f"{indent}Combine [{i}..{j}] -> {arr[i:j+1]}")

if __name__ == "__main__":
    data = [38, 27, 43, 3, 9, 82, 10]
    print("Data awal      :", data)
    print("\nProses Merge Sort (D&C):")
    print("-" * 45)

    merge_sort(data, 0, len(data) - 1)

    print("-" * 45)
    print("Data terurut :", data)

def merge_sort(arr, i, j, depth=0):
    indent = "  " * depth

    # basis: array dengan 1 elemen sudah terurut
    if i >= j:
        return

    # divide: hitung titik tengah pembagi
    k = (i + j) // 2
    print(f"{indent}Divide  [{i}..{j}] -> kiri [{i}..{k}], kanan [{k+1}..{j}]]")

    # conquer: urutkan bagian kiri secara rekursif
    merge_sort(arr, i, k, depth + 1)

    # conquer: urutkan bagian kanan secara rekursif
    merge_sort(arr, k + 1, j, depth + 1)

    # combine: gabungkan hasil pengurutan kedua bagian
    merge(arr, i, k, j)
    print(f"{indent}Combine [{i}..{j}] -> {arr[i:j+1]}")

if __name__ == "__main__":
    data = [38, 27, 43, 3, 9, 82, 10]
    print("Data awal      :", data)
    print("\nProses Merge Sort (D&C):")
    print("-" * 45)

    merge_sort(data, 0, len(data) - 1)

    print("-" * 45)
    print("Data terurut :", data)

```

Jawaban 1: Trace proses Divide-Conquer-Combine

Berikut trace verbose output program dengan data [38, 27, 43, 3, 9, 82, 10]:

Langkah	Operasi	Hasil
1	Divide [0..6] -> kiri [0..3], kanan [4..6]	
2	Divide [0..3] -> kiri [0..1], kanan [2..3]	
3	Divide [0..1] -> kiri [0..0], kanan [1..1]	
4	Combine [0..1]	[27, 38]
5	Divide [2..3] -> kiri [2..2], kanan [3..3]	
6	Combine [2..3]	[3, 43]
7	Combine [0..3]	[3, 27, 38, 43]
8	Divide [4..5] -> kiri [4..4], kanan [5..5]	
9	Combine [4..5]	[9, 82]
10	Combine [4..6]	[9, 10, 82]
11	Combine [0..6]	[3, 9, 10, 27, 38, 43, 82]

Jawaban 2: Output eksekusi

Data awal: [38, 27, 43, 3, 9, 82, 10]. Setelah proses Merge Sort D&C selesai, data terurut menjadi: [3, 9, 10, 27, 38, 43, 82]. Seluruh proses pembagian (Divide) dan penggabungan (Combine) ditampilkan verbose sehingga alur rekursi terlihat jelas dari kedalaman indentasi output.

IV. IMPLEMENTASI NYATA

Algoritma Merge Sort berbasis Divide and Conquer banyak diimplementasikan pada sistem pengurutan data berskala besar, termasuk pada sistem pemrosesan paralel. Jurnal berikut mendukung relevansi penerapannya.

[1] **Penulis:** Ketchaya, S. dan Rattanatanurak, A. (2023)

Judul: *"Parallel Multi-Deque Partition Dual-Deque Merge Sorting Algorithm Using OpenMP"*

Publikasi: Scientific Reports, Vol. 13, Nature Publishing Group. DOI: 10.1038/s41598-023-33583-4

Intisari: Makalah ini mengusulkan algoritma pengurutan paralel berbasis Divide and Conquer yang dikembangkan dari Quick Sort dan Merge Sort menggunakan OpenMP pada sistem memori bersama. Algoritma MPDMSort terbukti lebih efisien dibandingkan

implementasi sekuensial standar dalam pengurutan data biologis, saintifik, dan big data. Ini membuktikan bahwa prinsip D&C yang memecah masalah menjadi upa-masalah independen secara inheren mendukung paralelisme dan skalabilitas.

Link: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10115789/>

Kelebihan dalam konteks nyata:

- Kompleksitas $O(n \log n)$ yang konsisten membuat Merge Sort andal untuk pengurutan dataset besar seperti log server, data transaksi, dan rekaman medis.
- Sifat stabil Merge Sort menjamin urutan relatif elemen bernilai sama terjaga, penting dalam pengurutan multi-kunci.
- Prinsip D&C secara alami mendukung paralelisasi sehingga cocok untuk sistem multi-core modern.

Kekurangan dalam konteks nyata:

- Merge Sort membutuhkan memori tambahan $O(n)$ untuk proses penggabungan, sehingga kurang ideal untuk sistem dengan memori terbatas.
- Quick Sort memiliki worst case $O(n^2)$ apabila pemilihan pivot tidak tepat, sehingga perlu strategi pemilihan pivot yang baik seperti median-of-three untuk data dunia nyata.